

Git 컨벤션

🔍 구분

공유

1. 📌 이슈 (Issue) : "무엇을 해야 하는가?"

이슈는 모든 작업의 시작점입니다. 팀의 '할 일 목록(To-Do List)' 게시판이라고 생각하시면 됩니다.

- 목적:
 - 새로운 기능 제안, 버그 리포트, 코드 리팩토링 제안 등 모든 '할 일'을 등록하고 추적하기 위해 사용합니다.
- 작성 방법:
 - 제목: 작업을 명확하게 작성합니다. (예: 로그인 페이지 UI 구현, 회원가입 시 이메일 유효성 검사 오류)
 - 본문: (선택) 작업에 필요한 상세 요구 사항, 버그 재현 방법 등을 적습니다.
 - 담당자 (Assignees): 이 일을 누가 할지 지정합니다. (2분 중 한 명)
 - 라벨 (Labels): 이 이슈가 어떤 종류의 일인지, 얼마나 급한지 꼬리표를 붙입니다. (가장 중요!)
- 핵심:
 - 이슈를 생성하면 GitHub가 자동으로 번호(예: #12)를 부여합니다. 이 번호는 나중에 브랜치, 커밋, PR과 연결하는 핵심 고리가 됩니다.

라벨 (Labels) : 이슈의 "분류 꼬리표"

이슈를 효율적으로 관리하기 위해 제목에 [기능] 같은 접두사를 쓰는 대신 라벨을 사용합니다. 라벨은 여러 개를 동시에 붙일 수 있습니다.

리포지토리의 [Issues] -> [Labels] 탭에서 미리 만들어 두세요.

1. 🏷️ 우선순위 (Priority) 라벨

이 일이 얼마나 급한지 표시합니다.

- Priority: High (🔴 빨간색): 즉시 처리해야 할 일 (긴급 버그 등)
- Priority: Medium (🟡 주황색): 일반적인 작업
- Priority: Low (🟢 초록색): 급하지 않은 일 (문서 정리 등)

2. 📁 작업 종류 (Type) 라벨

이 일이 어떤 종류의 작업인지 구분합니다.

- Type: Feature (🔵 파란색): 새로운 기능 개발
- Type: Bug (🔴 빨간색): 버그 수정
- Type: Refactor (🟣 보라색): 코드 리팩토링 (기능 변경 없음)
- Type: Docs (⚪ 회색): README.md, 주석 등 문서 작업
- Type: Test (🟢 연두색): 테스트 코드 관련
- Type: UI (🎨 분홍색): CSS, 레이아웃 등 화면 수정
- Type: Chore (⚪ 회색): 라이브러리 업데이트, 빌드 설정 등 자잘한 잡무

2. 🌿 브랜치 (Branch) : "어디서 작업하는가?"

- 목적: 이슈(#12)에서 할당받은 작업을 수행하기 위한 독립적인 작업 공간을 만듭니다. (이슈 번호를 이름에 포함하는 것이 좋습니다.)
- 형식: 목적/이슈번호-내용
- 예시:

목적 (접두사)	예시 (이슈 번호 포함)	설명 (언제 사용?)
<code>feature/</code>	<code>feature/12-login-ui</code>	<code>Type: Feature</code> 라벨 이슈. (새로운 기능 개발)
<code>fix/</code>	<code>fix/13-email-validation-bug</code>	<code>Type: Bug</code> 라벨 이슈. (버그 수정)
<code>refactor/</code>	<code>refactor/14-user-service</code>	<code>Type: Refactor</code> 라벨 이슈. (코드 구조 개선)
<code>docs/</code>	<code>docs/18-add-readme-guide</code>	<code>Type: Docs</code> 라벨 이슈. (README, 주석 등 문서)
<code>test/</code>	<code>test/19-user-service-test</code>	<code>Type: Test</code> 라벨 이슈. (테스트 코드 작성/수정)
<code>ui/</code>	<code>ui/20-fix-button-css</code>	<code>Type: UI</code> 라벨 이슈. (CSS, HTML 등 화면 수정)
<code>chore/</code>	<code>chore/21-update-libraries</code>	<code>Type: Chore</code> 라벨 이슈. (빌드, 라이브러리 업데이트 등)

참고: 특별 케이스 (Hotfix)

목적 (접두사)	예시 (이슈 번호 없음)	설명 (언제 사용?)
<code>hotfix/</code>	<code>hotfix/critical-payment-error</code>	<code>main, dev</code> 브랜치에서 발생한 긴급 버그 수정 시. (이슈 기반이 아닌 <code>main, dev</code> 에서 바로 생성)

3. 📦 커밋 (Commits) : "작업 단계 기록"

- 목적:
브랜치에서 작업한 내용을 **잘게 쪼개어 의미 있는 단위로 저장**합니다. "그래서 뭘 했는지", 그리고 "왜 했는지(어떤 이슈 관련인지)"를 명확히 남깁니다.
- 형식 (강력 추천): `타입(#이슈번호): 제목 (명령형)`

1) 타입 (Type)

이 커밋이 무슨 성격인지 알려줍니다.

- `feat` : (필수) 새로운 기능 추가
- `fix` : (필수) 버그 수정
- `refactor` : 코드 리팩토링
- `docs` : 문서 수정 (README 등)
- `test` : 테스트 코드 관련
- `ui` : UI/CSS 등 화면 수정
- `chore` : 자잘한 수정, 빌드 파일 등

2) (#이슈번호)

이 커밋이 관련 있는 GitHub 이슈 번호를 괄호 안에 적습니다.

3) 제목 (Subject)

"~한다." (명령형)으로, 50자 이내로 간결하게 작성합니다.

📝 예시 시나리오

- * `#12` 번 이슈 ("로그인 기능 개발")**를 `feature/12-login` 브랜치에서 작업하는 경우:
- Good 👍 (좋은 예시):
 - `feat(#12): Add login page skeleton`
(12번 이슈 관련: 로그인 페이지 뼈대를 추가한다)
 - `feat(#12): Implement email validation`
(12번 이슈 관련: 이메일 유효성 검사를 구현한다)
 - `fix(#12): Correct password hashing logic`
(12번 이슈 관련: 비밀번호 해시 로직을 수정한다)
- Bad 👎 (나쁜 예시):
 - `fix` (뭘 고쳤는지, 몇 번 이슈인지 알 수 없음)

- `Added login function (#12)` (과거형 X)
- `#12 commit` (의미 없는 메시지)

4. 🚀 풀 리퀘스트 (PR) : "작업을 완료했으니 합쳐주세요"

- 목적: 특정 이슈(할 일)를 완료한 브랜치(커밋 묶음)를 `dev` 나 `main` 브랜치에 합쳐달라고(Merge) 정식으로 요청하는 것입니다.
- 형식: `[이슈번호] 커밋타입: 작업 요약 (명령형)`
- 예시:
 - `[#12] feat: Add login page UI`
 - `[#13] fix: Validate email on signup`
 - `[#14] refactor: Optimize UserService logic`
- 핵심: PR 본문에 `Closes #12` 라고 적으면, PR이 병합될 때 `#12` 이슈가 자동으로 'Closed' 처리됩니다.

한눈에 보는 워크플로우 비교

구분	📌 이슈 (Issue)	🌿 브랜치 (Branch)	📦 커밋 (Commit)	🚀 풀 리퀘스트 (PR)
역할	할 일 등록 (To-Do)	작업 공간 분리	작업 단계 기록	작업 완료 보고 & 병합 요청
네이밍	<code>[기능] 로그인 UI 구현</code>	<code>feature/12-login-ui</code>	<code>feat: Add login UI skeleton</code>	<code>[#12] feat: Add login page UI</code>
핵심	**"무엇"**을 할지 정의	**"이슈 번호"**로 연결	"어떻게" 했는지 기록	"완료" 및 리뷰 요청